

Multi-Agent Academic Operating System (MAAOS)

Abstract

Students today manage assignments, examinations, laboratory work, projects, notes, emails, and personal schedules across multiple applications. Most productivity tools only track deadlines and reminders, but they do not determine whether a student's workload is realistic or automatically adapt study plans when priorities change. This often leads to cognitive overload, poor time management, procrastination, and missed deadlines. The proposed Multi-Agent Academic Operating System (MAAOS) is an offline-first intelligent academic assistant that coordinates multiple AI agents to help students plan, prioritize, schedule, and complete their academic work efficiently.

The system consists of a Coordinator Agent, Priority Agent, Workload Agent, Time Scheduling Agent, Reminder Agent, Notes (RAG) Agent, Research Agent, OCR Agent, Analytics Agent, and an LLM Router Agent. The Coordinator Agent manages communication among all agents and combines their outputs into a single intelligent study plan. The Priority Agent ranks tasks based on deadlines, academic weightage, estimated effort, dependencies, and progress. The Workload Agent evaluates cognitive load by comparing planned study hours with the student's daily capacity. The Time Scheduling Agent creates realistic hourly study schedules while considering class timings, preferred study hours, breaks, and energy levels. The Reminder Agent sends intelligent notifications, while the Analytics Agent tracks productivity, study hours, and performance trends.

The platform incorporates Retrieval-Augmented Generation (RAG) using ChromaDB to answer questions from uploaded notes, OCR using Tesseract to extract information from syllabi, timetables, assignment sheets, and handwritten notes, voice commands, explainable AI, adaptive learning analytics, deadline-risk prediction, energy-aware scheduling, semester workload heatmaps, focus mode with Pomodoro sessions, gamification, a Digital Academic Twin, and a What-if Simulator that predicts the impact of changes in study plans.

To improve efficiency and reduce AI operating costs, the system uses a Hybrid LLM Router that intelligently switches between cloud-based language models and locally hosted models through Ollama. Simple operations such as task management, scheduling, reminders, and workload calculations are handled using deterministic algorithms without calling an LLM. If internet connectivity is unavailable or API usage limits are reached, the application automatically switches to local AI models and offline rule-based logic, ensuring uninterrupted operation.

As a Progressive Web Application (PWA), the platform supports offline task management, calendar access, reminders, downloaded notes, local AI-assisted scheduling, prompt caching, IndexedDB storage, queued AI requests, and automatic synchronization once connectivity is restored. The system is implemented using React PWA, FastAPI, CrewAI,

PostgreSQL/MySQL, ChromaDB, LiteLLM, Ollama, Tesseract OCR, SQLAlchemy, Docker, and JWT Authentication.

Team Member Module Allocation

Ann Maria: Responsible for the Priority and Task Management Module, including the Priority Agent, task management interface, backend APIs, CRUD operations, and intelligent task prioritization.

Shijo: Responsible for the Scheduling and Workload Module, including the Calendar interface, Time Scheduling Agent, Workload Agent, adaptive scheduling, workload balancing, and Pomodoro integration.

Jessica: Responsible for the AI Knowledge and Research Module, including document upload, ChromaDB, Retrieval-Augmented Generation (RAG), OCR, Research Agent, AI chat, LLM Router, and offline AI using Ollama.

Akshay: Responsible for the Coordinator and Analytics Module, including the Coordinator Agent, Reminder Agent, Analytics Dashboard, deadline-risk prediction, Digital Academic Twin, What-if Simulator, and notification management.

Shared Responsibilities

All team members will collaboratively contribute to frontend and backend integration, database integration, testing, debugging, GitHub version control, documentation, UI refinement, presentation preparation, and final deployment using Docker and an optional Jenkins CI/CD pipeline.